Artificial Intelligence
Lecture 10: Knowledge Representation and Ontologies

Prof. Dr. Pierre-Alexandre Murena

SoSe 2025

Outline

1 Knowledge representation

2 Description logics

3 Ontologies

The goal of knowledge representation

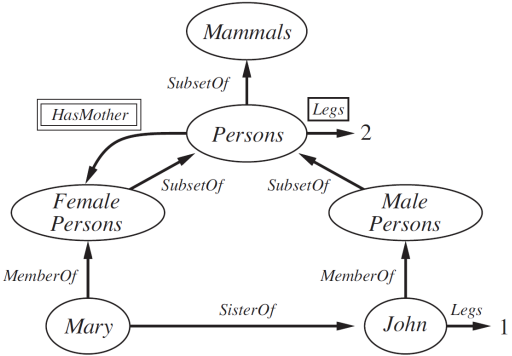
General goal: "Develop *formalisms* for providing *high-level descriptions* of the world that can be *effectively used* to build *intelligent applications*" [Brachman and Nardi, 2003]

- *formalism*: need for well-defined syntax and unambiguous semantics
- *high-level description*: only relevant aspects represented, easily interpretable descriptions
- *effectively used*: must provide practical and efficient reasoning tools
- *intelligent applications*: reasoning about the current knowledge and inferring implicit knowledge

A simple representation: Semantic networks

A **semantic network** is a graph for which:

- Vertices represent *objects* or *concepts*, and
- Edges represent semantic relations between concepts.

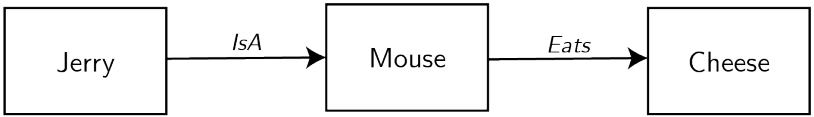


A simple representation: Semantic networks

In this semantic network, we represented:

- **objects:** e.g. Mary, John. They are referred to as **instances**;
- **classes of objects:** e.g. Mammals, Persons. They are referred to as **classes**.

Problem: It is not always easy to say what is a class and what is an object. For instance, is Chesse a class or an object?



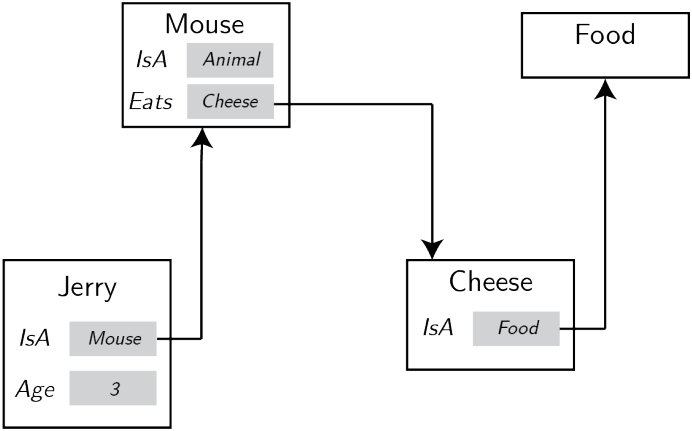
Beyond semantic networks: Frame Systems

Frame systems are a development of semantic networks proposed by Marvin Minsky, that allow to express the idea of **inheritance**.

As with semantic networks, a frame system is a graph, the vertices of which are **frames** and the edges of which are **relations**. Each frame can be either an **instance frame** or a **class frame**.

Each frame possesses a set of attributes, called **slots**, which have a value attached to them.

Beyond semantic networks: Frame Systems



Limitation of early knowledge representation languages

Even though the syntax of semantic networks and frame systems is simple, they do not offer a formal semantic.

Logical formalisms seem to be a reasonable alternative:

- Logics have a formal syntax and semantics.
- Various logics exist, with different expressivity (e.g. propositional, FO, modal etc)
- Optimised reasoning systems are available (inference)

Propositional logic for knowledge representation

The main advantage of **propositional logic** is that it is **decidable**: Given a knowledge base KB and a proposition ϕ, it is possible to show whether $KB \models \phi$ or $KB \not\models \phi$.

However, it is not convenient to describe complex domains with general properties and relations between objects.

First-order logic for knowledge representation

The semantics of first-order logic makes it a perfect candidate for knowledge representation, since it allows naturally to describe general properties and relations between objects.

However, first-order logic is too expressive. Intuitively, the more expressive the language is, the more complicated it is to show whether a statement is true.

First-order logic is known to be **undecidable**: there is no reasoning program that is guaranteed to produce correct results without the possibility of never terminating.

First-order logic for knowledge representation

The semantics of first-order logic makes it a perfect candidate for knowledge representation, since it allows naturally to describe general properties and relations between objects.

However, first-order logic is too expressive. Intuitively, the more expressive the language is, the more complicated it is to show whether a statement is true.

First-order logic is known to be **undecidable**: there is no reasoning program that is guaranteed to produce correct results without the possibility of never terminating.

Solution: Consider decidable fragments of first-order logic, called **description logics**

Outline

1 Knowledge representation

2 Description logics

3 Ontologies

Description logic

Description logic refers to a family of logic-based knowledge representation formalisms tailored towards representing **terminological knowledge**.

They are designed to make it easier to describe definitions and properties of categories.

Description logic

Description logic refers to a family of logic-based knowledge representation formalisms tailored towards representing **terminological knowledge**.

They are designed to make it easier to describe definitions and properties of categories.

Inference tasks:

- **Subsumption:** Checking if one category is a subset of another category;
- **Classification:** Checking whether an object belongs to a category;
- **Consistency:** Checking whether membership criteria are logically satisfiable

Plan

1 Knowledge representation

2 Description logics
- Syntax
- Semantics
- Expressing knowledge in description logic
- Some description logics

3 Ontologies

Basic building blocks

Description logics rely on 3 main building blocks:

Concept names: Atomic concepts that correspond to set of entities. They are equivalent to *unary predicates* in FOL.

Example: Mouse, Food, University, Person...

Basic building blocks

Description logics rely on 3 main building blocks:

Concept names: Atomic concepts that correspond to set of entities. They are equivalent to *unary predicates* in FOL.

Example: Mouse, Food, University, Person...

Role names: Relate two entities. They are equivalent to *binary predicates* in FOL.

Example: eats, ancestorOf, livesIn, isPresidentOf, directed...

Basic building blocks

Description logics rely on 3 main building blocks:

Concept names: Atomic concepts that correspond to set of entities. They are equivalent to *unary predicates* in FOL.

Example: Mouse, Food, University, Person...

Role names: Relate two entities. They are equivalent to *binary predicates* in FOL.

Example: eats, ancestorOf, livesIn, isPresidentOf, directed...

Individual names: denote particular entities. They are equivalent to *constants* in logic.

Example: jerry, tuhh, georgeMiller, ...

Constructors

Constructors are operations used to form concept terms. They can be:

- **Logical constructors:** Intersection (conjunction) of concepts, Union (disjunction) of concepts, complement (negation) of concepts, universal restriction ($\forall$)
- **Expressive number restrictions:** restrictions of the number of states (used to express for instance notions like "people with less than 3 children").
- **Role constructors:** Conditions on roles

Example: The class of people living in Hamburg and having at leat 2 children is:

$$People \sqcap livesIn.(hamburg) \sqcap \geq 2 hasChild.\top$$

Conjunction, disjunction, negation

Conjunction: $C \sqcap D$, entities that belong to **both** C and D.

$$ComputerScientist \sqcap Musician$$

Disjunction: $C \sqcup D$, entities that belong to **either** C or D.

$$Cat \sqcup Dog$$

Negation: $\neg C$, entities that are not in C.

$$\neg Director$$

Remark: this class will contain any entity that is not a director, including non-human entities...

Existential restrictions

Given a role r (i.e. a binary relation) and a concept C, we denote by $\exists r.C$ the class of entities that are related by r to an entity in C:

Examples:

1. Those who directed a comedy:

$$\exists directed.Comedy$$

2. Those whose child owns a dog:

$$\exists parentOf.(\exists owns.Dog)$$

Universal restrictions

Given a role r (i.e. a binary relation) and a concept C, we denote by $\forall r.C$ the class of entities that are only related by r to entities from C.

Examples: Those who own dogs only:

$$\forall own.Dog$$

Universal restrictions

Given a role r (i.e. a binary relation) and a concept C, we denote by $\forall r.C$ the class of entities that are only related by r to entities from C.

Examples: Those who own dogs only:

$$\forall own.Dog$$

Remark: This notation can be misleading! Trivially, someone who doesn't own anything still belongs to this class. To restrict to the category of owners, one must specificy it explicitly:

$$(\forall own.Dog) \sqcap (\exists own.\top)$$

Number restrictions

Given a role r (i.e. a binary relation), a concept C and a non-negative integer k, we denote by $\geq kr.C$ (resp. $\leq kr.C$) the class of entities that are related by r to at least (resp. at most) k entities from C:

1. Those who have directed at least 3 comedies

$$\geq 3directed.Comedy$$

2. Those who have less than 4 children:

$$\leq 4parentOf.\top$$

Plan

1 Knowledge representation

2 Description logics
- Syntax
- Semantics
- Expressing knowledge in description logic
- Some description logics

3 Ontologies

Interpretations

The semantic of description logic is very similar to the semantic of first-order logic.

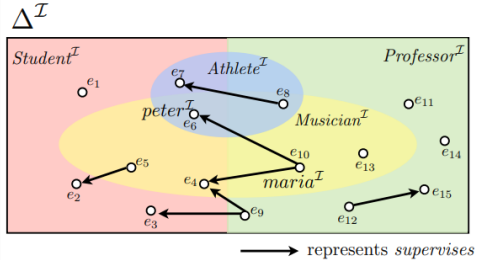
An interpretation $\mathcal{I}$ is a tuple $\mathcal{I} = \langle \mathcal{D}^{\mathcal{I}}, .^{\mathcal{I}} \rangle$, where:

- $\mathcal{D}^{\mathcal{I}}$ is a non-empty set called **domain of interpretation**
- $.^{\mathcal{I}}$ is a function which maps:
  - Individual names a to an element $a^{\mathcal{I}}$ of $\mathcal{D}^{\mathcal{I}}$;
  - Concept names A to a unary relation $A^{\mathcal{I}} \subseteq \mathcal{D}^{\mathcal{I}}$;
  - Role names r to a binary relation $r^{\mathcal{I}} \subseteq \mathcal{D} \times \mathcal{D}^{\mathcal{I}}$.

Example

Interpretation $\mathcal{I} = (\Delta^{\mathcal{I}}, \cdot^{\mathcal{I}})$ where

$$\Delta^{\mathcal{I}} = \{e_1, e_2, \ldots, e_{15}\}$$
$$Student^{\mathcal{I}} = \{e_1, \ldots, e_7\}$$
$$Musician^{\mathcal{I}} = \{e_4, e_5, e_6, e_8, e_{10}, e_{13}\}$$
$$supervises^{\mathcal{I}} = \{(e_5, e_2), (e_8, e_7), (e_{10}, e_4), (e_{10}, e_6), (e_9, e_3), (e_9, e_4), (e_{12}, e_{15})\}$$
$$Professor^{\mathcal{I}} = \{e_8, \ldots, e_{15}\}$$
$$Athlete^{\mathcal{I}} = \{e_6, e_7, e_8\}$$
$$maria^{\mathcal{I}} = e_{10}$$
$$peter^{\mathcal{I}} = e_6$$



Semantic of constructors

As for propositional and first-order logic, we now need to extend the interpretation to constructors:

$$
\begin{aligned}
\top^{\mathcal{I}} &= \mathcal{D}^{\mathcal{I}} \\
(C \sqcap D)^{\mathcal{I}} &= C^{\mathcal{I}} \cap D^{\mathcal{I}} \\
(C \sqcup D)^{\mathcal{I}} &= C^{\mathcal{I}} \cup D^{\mathcal{I}} \\
(\neg C)^{\mathcal{I}} &= \mathcal{D}^{\mathcal{I}} \setminus C^{\mathcal{I}} \\
(\exists r.C)^{\mathcal{I}} &= \{u \mid \text{there exists } v \text{ such that } (u,v) \in r^{\mathcal{I}} \text{ and } v \in C^{\mathcal{I}}\} \\
(\forall r.C)^{\mathcal{I}} &= \{u \mid \text{for every } v \text{ such that } (u,v) \in r^{\mathcal{I}} \text{ we have } v \in C^{\mathcal{I}}\} \\
(\geq kr.C)^{\mathcal{I}} &= \{u \mid \text{at least } nv \text{ such that } (u,v) \in r^{\mathcal{I}} \text{ and } v \in C^{\mathcal{I}}\}
\end{aligned}
$$

Plan

1 Knowledge representation

2 Description logics
- Syntax
- Semantics
- Expressing knowledge in description logic
- Some description logics

3 Ontologies

ABox and TBox

We often distinguish two kinds of knowledge:

- **General domain knowledge (ontology):** finite set of axioms about the domain (e.g. "mice eat cheese"). These axioms are called **TBox**.
- **Factual knowledge about particular individuals:** finite set of assertions (e.g. "Jerry is a mouse"). These assertions are called **ABox**.

Axioms about concepts

Axioms about concepts are the form:

$$C \sqsubseteq D$$

which means that every member of C is also a member of D. We call such axioms **concept inclusion**.

Example:

$$Professor \sqsubseteq \exists worksIn.University$$

From concept inclusion, we define the notion of **concept equivalence**:

$$C \equiv D \text{ if and only if } C \sqsubseteq D \text{ and } D \sqsubseteq C$$

Satisfaction of axioms and assertions

An interpretation $\mathcal{I}$ **satisfies**:

- a concept inclusion $C \sqsubseteq D$ if $C^{\mathcal{I}} \subseteq D^{\mathcal{I}}$
- a concept equivalence $C \equiv D$ if $C^{\mathcal{I}} = D^{\mathcal{I}}$
- an assertion $C(a) \sqsubseteq D$ if $a^{\mathcal{I}} \in C^{\mathcal{I}}$
- an assertion $r(a, b) \sqsubseteq D$ if $(a^{\mathcal{I}}, b^{\mathcal{I}}) \in r^{\mathcal{I}}$

Satisfaction of axioms and assertions

An interpretation $\mathcal{I}$ **satisfies**:

- a concept inclusion $C \sqsubseteq D$ if $C^{\mathcal{I}} \subseteq D^{\mathcal{I}}$
- a concept equivalence $C \equiv D$ if $C^{\mathcal{I}} = D^{\mathcal{I}}$
- an assertion $C(a) \sqsubseteq D$ if $a^{\mathcal{I}} \in C^{\mathcal{I}}$
- an assertion $r(a, b) \sqsubseteq D$ if $(a^{\mathcal{I}}, b^{\mathcal{I}}) \in r^{\mathcal{I}}$

Models:

- $\mathcal{I}$ is a model of a TBox $\mathcal{T}$ iff $\mathcal{I}$ satisfies all axioms in $\mathcal{T}$.
- $\mathcal{I}$ is a model of an ABox $\mathcal{A}$ iff $\mathcal{I}$ satisfies all axioms in $\mathcal{A}$.
- $\mathcal{I}$ is a model of a KB $(\mathcal{T}, \mathcal{A})$ iff $\mathcal{I}$ is a model of $\mathcal{T}$ and $\mathcal{A}$.

Satisfiability and entailment

Satisfiability:

- A concept C is **satisfiable** w.r.t. a TBox $\mathcal{T}$ if there exists a model $\mathcal{I}$ of $\mathcal{T}$ with $C^{\mathcal{I}} \neq \emptyset$
- A knowledge base $(\mathcal{T}, \mathcal{A})$ is satisfiable if $(\mathcal{T}, \mathcal{A})$ has at least one model.

Satisfiability and entailment

Satisfiability:

- A concept C is **satisfiable** w.r.t. a TBox $\mathcal{T}$ if there exists a model $\mathcal{I}$ of $\mathcal{T}$ with $C^{\mathcal{I}} \neq \emptyset$
- A knowledge base $(\mathcal{T}, \mathcal{A})$ is satisfiable if $(\mathcal{T}, \mathcal{A})$ has at least one model.

Entailment:

- A TBox $\mathcal{T}$ entails an axiom α (denoted $\mathcal{T} \models \alpha$ if every model of $\mathcal{T}$ satisfies α.
- A KB $(\mathcal{T}, \mathcal{A})$ entails an ABox assertion α (denoted $KB \models \alpha$) if every model of $(\mathcal{T}, \mathcal{A})$ satisfies α.

Plan

1 Knowledge representation

2 Description logics
- Syntax
- Semantics
- Expressing knowledge in description logic
- Some description logics

3 Ontologies

Defining a description logic

The presented syntax and semantics are very general. Depending on the intended expressivity of the description logic, one may not want to include all constructors.

Three important families of DLs are:

- **Attributive language** $\mathcal{AL}$: allows atomic negation, concept intersection, universal restriction ($\forall$), limited existential quantification ($\exists$).
- **Frame-based language** $\mathcal{FL}$: concept intersection, universal restrictions, limited existential quantification, role restriction
- **Existential language** $\mathcal{EL}$: concept intersection, existential restrictions

The $\mathcal{ALC}$ logic

The $\mathcal{ALC}$ logic has been historically an important description logic. It corresponds to a language based on all the constructors presented so far.

The main limitation of $\mathcal{ALC}$ is that it cannot express role transitivity: if a role relates A and B, and B and C, then it should relate A and C.

Outline

1 Knowledge representation

2 Description logics

3 Ontologies

Plan

1 Knowledge representation

2 Description logics

3 Ontologies
- What are ontologies?
- RDFS
- Web Ontology Language (OWL)

Ontologies: definition and specification

An **ontology** is a formal specification of the knowledge of a domain, making it amenable to machine processing.

It consists of:

- the **terminology** of the domain;
- **semantic relationship** between terns

Ontologies: definition and specification

An **ontology** is a formal specification of the knowledge of a domain, making it amenable to machine processing.

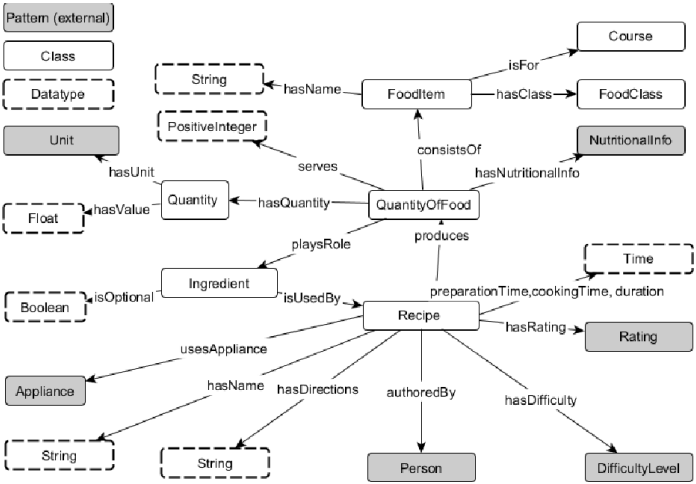
It consists of:

- the **terminology** of the domain;
- **semantic relationship** between terns

Ontologies can be used to:

- **Standardize** the terminology of an application domain: *constraining the meaning of terms, adopting a common vocabulary*
- Present an **intuitive and unified view of the data**
- **Support automated reasoning**

An example of ontology



The language of ontologies

In order to describe knowledge properly, ontologies must be formulated in a well-chosen language:

Natural language is not a good candidate: too imprecise and ambiguous, and not machine-interpretable.

Formal logic is an excellent mathematical franework to represent knowledge. In practice, it does not interface perfectly well with a machine.

The language of ontologies

In order to describe knowledge properly, ontologies must be formulated in a well-chosen language:

Natural language is not a good candidate: too imprecise and ambiguous, and not machine-interpretable.

Formal logic is an excellent mathematical franework to represent knowledge. In practice, it does not interface perfectly well with a machine.

Standardized languages: OWL, RDFS

Plan

1 Knowledge representation

2 Description logics

3 Ontologies
- What are ontologies?
- RDFS
- Web Ontology Language (OWL)

Resource Description Framework

Resource Description Framework Schema (RDFS) is a simple schema language commonly used in the Semantic Web technology. It is elementary and very easy to use.

RDF models the domain with triples: (subject, predicate, object).

- (Germany, partOf, Europe)
- (Germany, isA, Country)
- (Germany, hasCapital, Berlin)

Resource Description Framework Schema

The knowledge is then structured as a graph (similar to semantic networks), in which:

- The nodes can be:
  - **Resource nodes**: correspond to an entity or a class of entities
  - **Literal nodes**: associated to literal values (strings, numbers etc). For instance, a name, a quantity etc.
  - **Blank nodes**: *more advanced topic, not covered in this lecture!*
- The edges correspond to a RDF connecting the subject node to the object node.

Remark: The predicates can represent any form of relation between nodes: parts of, inheritance, logical relation...

Unicity of objects

Objects have to be unique: they are then identified by **Uniform Resource Identifiers** (URI). These URIs are meant to make the vocabulary consistent across databases

URIs often correspond to URLs (but not always).

Limitations of RDFS

Logical consistency: RDFS present no real constraint on what is represented. For instance, consider the following RDFS:

```
MyExample:Labrador rdf:Type rdfs:Class
MyExample:Fido rdf:Type MyExample:Labrador
MyExample:Labrador rdf:Type MyExample:BigDog
MyExample:BigDog rdf:Type rdfs:Class
```

In this example, `MyExample:Labrador` is at the same type a class (first line) and an instance of the class `MyExample:BigDog` (lines 3-4).

In practical applications, we want unambiguous knowledge representation.

Limitations of RDFS

Limited expressivity: The expressivity of RDFS is somehow limited to the expressivity of a frame system. In particular, it does not allow for complex constructors as in description logics.

For instance, it cannot express that a mother is a parent who is a woman.

Limitations of RDFS

Limited expressivity: The expressivity of RDFS is somehow limited to the expressivity of a frame system. In particular, it does not allow for complex constructors as in description logics.

For instance, it cannot express that a mother is a parent who is a woman.

Representation, but no reasoning: RDFS is a representation language, the goal of which is *only* to represent domains. It is not developed to be able to reason about it.

Plan

1 Knowledge representation

2 Description logics

3 Ontologies
- What are ontologies?
- RDFS
- Web Ontology Language (OWL)

OWL

Web Ontology Language (OWL) is a data language based on RDFS, with the expressivity of description logics. However, unlike RDF, OWL guarantees:

- Expressive and flexible data modeling
- Efficient automated reasoning

Expressivity of OWL

OWL allows for everything that was already possible with RDFS, but even more!

- Non-unicity of representations: `owl:sameAs`. For instance: `ex:ElvisPresley owl:sameAs ex:TheKingOfRockNRoll`
- Properties of relations: inverse (`owl:inverseOf`), transitivity (`owl:TransitiveProperty`) etc. For instance `:parentOf owl:inverseOf :childOf`.
- Class equivalence: `owl:equivalentClass`. For instance `:Human owl:equivalentClass :Person`
- Constructors of description logic. For instance:

```
:Mother rdfs:subClassOf [ owl:intersectionOf ( :Female :Parent ) ]
```

To know more...

Description logics:

- Baader, F., et al. (2008). Description logics.

RDFS / OWL:

- Arenas, M., et al. (2009). Foundations of RDF databases.
- Antoniou, G., & Harmelen, F. V. (2009). Web ontology language: OWL.
- Hogan A.: Lecture on Web Ontology Language